

DAT42-2

# Coding & Working with LLMs

---

Albert School — 16 hours

# Preface

You already know how to write Python that reads data, calls a function, handles an error, and talks to a web API over HTTP (DAT12-2). This course adds one new kind of service to the ones you can already call: a **large language model** (LLM). Nothing here asks you to understand how the model is built or trained — that lives in Math S4 and DAT62-1. Here, the LLM is a tool you call from Python, exactly like the weather API you called with `requests`, except that its input and output are natural language and its behaviour is probabilistic rather than deterministic.

That last difference is the whole reason this course exists. A lookup table returns the same answer every time; an LLM samples its answer from a distribution, can be steered by how you phrase the request, costs money per word, and will sometimes produce confident nonsense. Using one well means understanding just enough about **how it produces text** to call the API intelligently, writing prompts that reliably get the output you want, measuring whether one prompt beats another, computing similarity between texts, and — just as important — recognising the situations where reaching for an LLM is the wrong engineering decision. We will build each of these skills from a concrete example before stating the general rule.

# Contents

|                                              |    |
|----------------------------------------------|----|
| Preface .....                                | 2  |
| 1 What an LLM does, practically .....        | 2  |
| 1.1 Temperature and top-p .....              | 3  |
| 2 Calling LLM APIs from Python .....         | 3  |
| 2.1 Request and response structure .....     | 4  |
| 2.2 Error handling and retry logic .....     | 5  |
| 2.3 Token counting and cost estimation ..... | 5  |
| 3 Prompt engineering .....                   | 6  |
| 3.1 Zero-shot .....                          | 6  |
| 3.2 Few-shot .....                           | 6  |
| 3.3 Chain-of-thought .....                   | 7  |
| 3.4 Structuring the output .....             | 7  |
| 4 Systematic prompt evaluation .....         | 7  |
| 5 Embeddings and semantic similarity .....   | 8  |
| 5.1 Cosine similarity .....                  | 8  |
| 5.2 Basic semantic search .....              | 9  |
| 6 LLM failure modes .....                    | 9  |
| 6.1 When LLMs are inappropriate .....        | 10 |
| 7 Cost-effectiveness .....                   | 10 |

## 1 What an LLM does, practically

Suppose you type into a model the words *The capital of France is* and ask it to continue. Internally, the model does not “look up” the answer. It produces, for every possible next chunk of text, a probability: *Paris* might get 0.92, *a* might get 0.01, *located* 0.005, and so on across tens of thousands of candidates. It then **picks one** according to those probabilities, appends it, and repeats the whole process with the longer text as input. Generating a sentence is this loop run dozens of times.

These chunks are called **tokens**. A token is roughly a word or a word-piece: **Paris** is one token, but a rare word like **tokenisation** might be split into **token** + **isation**. You do not choose

the tokenisation — the model’s tokenizer does — but you pay per token and are limited by a maximum number of tokens, so it is worth knowing the unit exists.

**Definition 1 (Next-token prediction)** An LLM models the probability of the next token given all preceding tokens. To generate text it repeatedly samples one token from this predicted distribution and appends it to the sequence, then predicts again.

## 1.1 Temperature and top-p

Because the next token is **sampled** rather than chosen by a fixed rule, you get a knob: how adventurous should the sampling be? Two parameters control this.

Imagine the model’s predicted probabilities for the next token drawn as bars. **Temperature** reshapes those bars before sampling. At a temperature near 0, the tallest bar is exaggerated until it is almost the only option — the model becomes nearly deterministic and tends to repeat itself. At a higher temperature such as 1.0, the bars are flattened, so less likely tokens get a real chance — the output becomes more varied and surprising (and, pushed too far, incoherent).

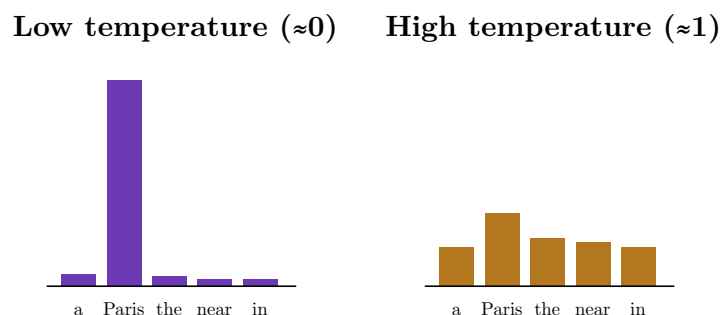


Figure 1: The same predicted distribution, reshaped by temperature. Low temperature sharpens it toward the single most likely token; high temperature flattens it so alternatives become competitive.

**Top-p** (also called nucleus sampling) cuts the distribution off instead of reshaping it. With `top_p = 0.9`, you sort the candidate tokens from most to least probable, walk down the list adding up their probabilities, and stop once the running total reaches 90%. Only the tokens above that cut are eligible; the long tail of unlikely tokens is discarded entirely. This caps how strange the output can get while still allowing variety among the plausible options.

**Remark** A useful default intuition: lower temperature / lower top-p for tasks with one correct answer (extraction, classification, code that must run); higher values for tasks where you want diversity (brainstorming, creative writing). Most production APIs default to moderate values that work well enough that you rarely need to touch them.

This is the entire conceptual basis you need to use the API intelligently. How the model acquires these probabilities — the architecture and the training — is out of scope.

## 2 Calling LLM APIs from Python

You already called a web API in DAT12-2: send an HTTP request, get JSON back, parse it. An LLM API is the same shape. The only practical change is that you use the provider’s **official SDK** (a Python library that wraps the HTTP calls) instead of writing `requests` calls by hand, and that the “input” is a conversation rather than a query string.

The examples below use Anthropic’s API. OpenAI’s is structurally identical — same idea of messages, roles, and a token-counted response — so everything transfers.

First, install the SDK into your virtual environment (the `venv` workflow from DAT12-2):

```
pip install anthropic
```

Then a first call:

```
import anthropic

client = anthropic.Anthropic() # reads ANTHROPIC_API_KEY from the environment

response = client.messages.create(
    model="claude-opus-4-8",
    max_tokens=1024,
    messages=[
        {"role": "user", "content": "What is the capital of France?"}
    ],
)
print(response.content[0].text)
```

**Common pitfall** Never paste your API key directly into the code (`Anthropic(api_key="sk-...")`). A key committed to a repository is a leaked credential — anyone who finds it can spend your money. Store it in an environment variable (here `ANTHROPIC_API_KEY`) and let the SDK read it, exactly as you would keep a database password out of source code.

## 2.1 Request and response structure

Think of the request as describing a conversation and the response as the model’s next turn in it.

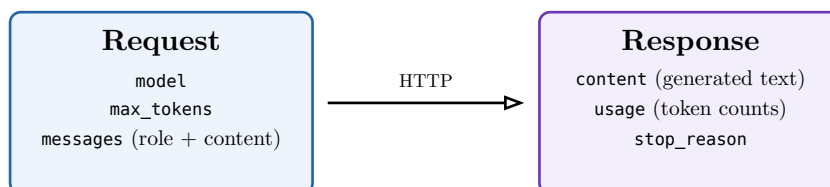


Figure 2: The request carries the model name, an output-length cap, and the conversation so far. The response carries the model’s reply plus accounting metadata.

- The **request** contains the model identifier, a `max_tokens` cap that bounds how long the output may be, and a list of `messages`. Each message has a `role` (“user” or “assistant”) and its `content`.
- The **response** is an object whose `content` holds the generated text and whose `usage` field reports how many input and output tokens the call consumed.

**Common pitfall** The API is **stateless**: it does not remember previous calls. To hold a multi-turn conversation you must resend the entire history — every prior user and assistant message — on each new call. Forgetting this is the most common beginner mistake; the model appears to have “amnesia” because you only sent it the latest message.

A short conversation therefore looks like this:

```

messages = [
    {"role": "user", "content": "My name is Alice."},
    {"role": "assistant", "content": "Nice to meet you, Alice."},
    {"role": "user", "content": "What is my name?"}, # needs the history above
]

```

## 2.2 Error handling and retry logic

Network calls fail, and you learned the defensive reflex in DAT12-2: wrap them in `try/except`. LLM calls add one nuance — **which** failures are worth retrying.

- **Transient** failures — rate limits (HTTP 429) and server errors (HTTP 5xx) — are temporary. The same request may well succeed a moment later, so retry it.
- **Permanent** failures — an invalid API key (401), a malformed request (400) — will fail identically no matter how many times you resend. Do not retry; fix the request.

When you do retry, do not hammer the server immediately. Use **exponential backoff**: wait a delay that grows after each attempt (1s, then 2s, then 4s...), giving the overloaded service room to recover.

```

import time
import anthropic

client = anthropic.Anthropic()

def call_with_retry(messages, max_retries=5):
    for attempt in range(max_retries):
        try:
            return client.messages.create(
                model="claude-opus-4-8",
                max_tokens=1024,
                messages=messages,
            )
        except (anthropic.RateLimitError, anthropic.InternalServerError):
            if attempt == max_retries - 1:
                raise # give up after the last attempt
            time.sleep(2 ** attempt) # 1s, 2s, 4s, 8s, ...

```

**Remark** The official SDKs already retry transient errors with backoff automatically, so in practice you often need no retry code at all. The snippet above is shown so you understand the logic the SDK is applying on your behalf — and so you can implement it yourself against any API that lacks an SDK.

## 2.3 Token counting and cost estimation

You are billed per token, and separately for tokens you **send** (input) and tokens the model **generates** (output) — output is the more expensive of the two. Because cost scales with token count, estimating it before sending is straightforward.

You can count the tokens of a prompt before making the (paid) generation call:

```

count = client.messages.count_tokens(
    model="claude-opus-4-8",
    messages=[{"role": "user", "content": "Hello"}],
)
print(count.input_tokens)

```

After a real call, `response.usage` reports `input_tokens` and `output_tokens` actually consumed.

Prices are quoted per **million** tokens (written “/ MTok”) and differ by model. A larger, more capable model costs more per token; a smaller one costs less. Indicative current prices:

| Model                                | Input / MTok | Output / MTok |
|--------------------------------------|--------------|---------------|
| Claude Opus 4.8 (most capable)       | \$5.00       | \$25.00       |
| Claude Sonnet 4.6 (balanced)         | \$3.00       | \$15.00       |
| Claude Haiku 4.5 (fastest, cheapest) | \$1.00       | \$5.00        |

Table 1: Indicative per-token prices. The cheaper models are several times less expensive — choosing the smallest model that does the job is the main cost lever.

A worked cost calculation. Suppose a call sends 1,000 input tokens and generates 500 output tokens on Opus 4.8, at \$5.00 / MTok input and \$25.00 / MTok output:

$$\frac{1000}{1000000} \times \$5.00 + \frac{500}{1000000} \times \$25.00 = \$0.005 + \$0.0125 = \$0.0175.$$

Less than two cents for one call — but multiply by a million calls a month and it is \$17,500, which is why model choice and prompt length matter at scale. We return to when that spend is justified in Section 7.

## 3 Prompt engineering

The **prompt** is the text you send the model. Because the model’s output is steered by its input, the way you phrase a request materially changes the answer you get. Prompt engineering is the craft of phrasing prompts to reliably get the output you want. It is genuinely empirical: small wording changes can have large effects, so you try, observe, and adjust.

We will introduce each technique with a task: **classify the sentiment of a product review as positive or negative**.

### 3.1 Zero-shot

The simplest approach: just state the task, with no examples.

Classify the sentiment of this review as positive or negative: "The food was cold."

This often works for easy tasks. When it does, stop here — it is the cheapest and shortest prompt.

### 3.2 Few-shot

When zero-shot is unreliable — say the model keeps answering in full sentences when you want a single word, or mishandles sarcasm — **show** it a few worked examples first. The model infers the pattern (including the exact output format) from your demonstrations.

Review: "Loved it!" -> positive

Review: "Terrible service." -> negative

Review: "The food was cold." ->

**Remark** “Few-shot” means a few examples (typically two to five); “zero-shot” means none. The examples teach both **what** to do and **how to format the answer** — note how the layout above implicitly tells the model to reply with one lowercase word.

### 3.3 Chain-of-thought

For tasks needing several reasoning steps — arithmetic, logic, multi-part questions — ask the model to **reason step by step before answering**. Writing out intermediate steps measurably improves accuracy, because each generated step becomes context for the next.

A shop sells pens at 3 for \$2. How much do 12 pens cost?

Think step by step, then give the answer.

**Common pitfall** Chain-of-thought trades tokens (and therefore cost and latency) for accuracy. Do not switch it on reflexively: on a simple classification it just makes the model ramble and bills you for the rambling. Reserve it for problems that genuinely have multiple steps.

### 3.4 Structuring the output

Three further techniques shape the **form** of the answer rather than the reasoning:

- **Persona assignment** — tell the model what role to adopt, which sets vocabulary and depth: "You are an expert tax accountant."
- **Output format specification** — state the exact format you need, so you can parse it reliably: "Reply with valid JSON only, with keys 'sentiment' and 'confidence'."
- **Constraint injection** — add explicit limits: "Answer in at most two sentences. Do not mention prices."

**Worked example — Combining techniques.** A robust production prompt for our sentiment task might assign a persona ("You are a precise text classifier"), give two few-shot examples, and specify the output format ("Reply with exactly one word: positive or negative"). Each technique addresses a different failure you observed while testing — that is the real workflow: start simple, add structure only where the output falls short.

## 4 Systematic prompt evaluation

Once you have two candidate prompts — say zero-shot versus few-shot — how do you know which is **better**? Eyeballing one or two outputs is unreliable: you might have looked at the examples where they happen to agree. You need to measure.

The mistake to avoid is judging on a single impressive (or disappointing) example. A prompt that nails one case may fail systematically on others. Evaluation means running both prompts over the **same set** of inputs and scoring every output against the **same** explicit standard.

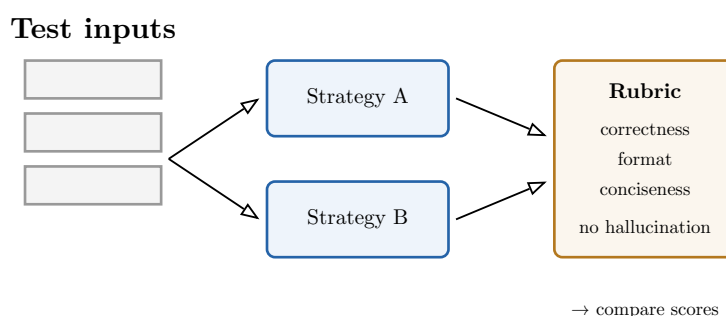


Figure 3: Both prompting strategies run on the same inputs; every output is scored against the same rubric; the strategies are then compared on their total scores.

The procedure:

1. Define a set of **criteria** the output must satisfy — for our task: correct label, single word, lowercase, no extra commentary.
2. Build a **rubric** that says, for each criterion, what counts as acceptable. This turns a vague “is it good?” into a checklist anyone could apply identically.
3. Run each prompting strategy on the **same** set of inputs (a dozen reviews, say, including hard cases like sarcasm).
4. Score each output against the rubric.
5. Compare the strategies on their aggregate scores, and conclude which won and why.

The deliverable for this course is exactly that: a comparative evaluation of two prompting strategies on one task, scored against one rubric, ending in a justified conclusion. The justification matters as much as the number — “few-shot scored higher because it fixed the format errors zero-shot made on sarcastic reviews” is the kind of statement you are aiming for.

## 5 Embeddings and semantic similarity

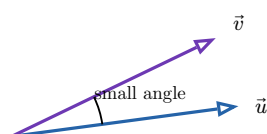
So far the model produced text. Sometimes you instead want to **compare** texts by meaning — to ask “are these two sentences about the same thing?” even when they share no words. That is what embeddings give you.

Consider “a dog on the beach” and “a puppy by the sea.” No word is shared, yet they mean almost the same thing. A keyword search would call them unrelated. An **embedding** fixes this by mapping each text to a vector of numbers positioned so that similar meanings land near each other in space.

**Definition 2 (Embedding)** An embedding is a fixed-length vector of numbers representing a piece of text. A sentence embedding model maps text to such a vector so that texts with similar meaning have vectors that point in similar directions.

### 5.1 Cosine similarity

How do we measure “point in similar directions”? By the angle between the two vectors. Two vectors pointing the same way have a small angle; unrelated ones are roughly perpendicular; opposite meanings point apart. We summarise the angle with its cosine.



similar meaning  $\Rightarrow$  cosine near 1

Figure 4: Two embedding vectors. The smaller the angle between them, the closer the cosine is to 1, and the more similar the texts’ meanings.

**Definition 3 (Cosine similarity)**

$$\text{cosine}(\vec{u}, \vec{v}) = \frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\| \|\vec{v}\|}.$$

It ranges from  $-1$  (opposite) through  $0$  (unrelated) to  $1$  (same direction, i.e. maximally similar in meaning).



The numerator is the dot product (multiply the vectors component-wise and add); the denominator divides out the vectors' lengths so that only their **direction** — the meaning — matters, not how long the text is.

```
from sentence_transformers import SentenceTransformer
import numpy as np

model = SentenceTransformer("all-MiniLM-L6-v2")
u, v = model.encode(["a dog on the beach", "a puppy by the sea"])
cosine = u @ v / (np.linalg.norm(u) * np.linalg.norm(v))
print(cosine) # close to 1: the two sentences mean nearly the same thing
```

**Remark** Computing an embedding is usually a separate, much cheaper model than the text-generating LLM, and many embedding models run locally with no API call at all (as above). Don't reach for a full LLM call to ask "are these similar?" when a cosine comparison answers it for free.

## 5.2 Basic semantic search

Embeddings give you search-by-meaning almost immediately. To find which of several documents best answers a query: embed the query and every document, then return the documents whose embeddings have the highest cosine similarity to the query's. Because matching is on meaning rather than shared keywords, a query for "how do I reset my password" can surface a document titled "recovering account access" even though the words differ.

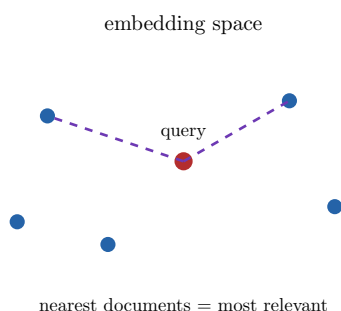


Figure 5: Query and documents placed in the same embedding space. The documents nearest the query (highest cosine similarity) are the most semantically relevant.

## 6 LLM failure modes

An LLM is fluent and confident even when it is wrong — that is precisely what makes it dangerous to trust blindly. Knowing the characteristic ways it fails is what lets you tell when it is the wrong tool.

- **Hallucination.** The model produces fluent, confident text that is factually wrong or entirely fabricated — an invented citation, a non-existent API method, a made-up statistic. The danger is that hallucinations look exactly like correct answers; nothing in the tone signals the difference. If you asked for a legal case and got a realistic-looking but fictional one, you have seen this.
- **Sycophancy.** The model tends to agree with the user and tell them what they seem to want to hear. Ask "this code is correct, right?" and it is more likely to validate you than to find the bug. Phrase questions neutrally to avoid leading it.
- **Prompt injection.** When you feed the model untrusted external text — a web page, a user-submitted document, an email — that text may itself contain instructions ("ignore

your previous instructions and ...”). The model can obey them. This is a genuine security vulnerability, not a curiosity: any time the prompt includes content you do not control, treat it as a potential attack vector.

- **Context window limits.** The model can only attend to a bounded number of tokens at once — its **context window**. Text beyond that limit cannot be processed; it must be truncated, split, or summarised, and information will be lost. A 500-page document does not simply fit.

**Common pitfall** These failure modes do not announce themselves. A hallucinated fact and a correct fact arrive in the same confident prose. This is why output that matters must be **verified** against a trustworthy source rather than taken on faith.

## 6.1 When LLMs are inappropriate

Putting the failure modes together, an LLM is a poor choice when:

- the task demands guaranteed factual accuracy and you have no way to verify the output (hallucination risk);
- the prompt must include untrusted external text **and** the resulting action is sensitive, e.g. sending an email or running a command (prompt-injection risk);
- the input is larger than the context window and cannot be meaningfully split;
- a simpler, cheaper, deterministic method — a database lookup, a regular expression, a business rule — would solve the task reliably. An LLM is overkill for checking whether a string is a valid email address.

## 7 Cost-effectiveness

The final judgement is an engineering one: is an LLM call worth it here? You now have both halves of the answer — the cost (from token counting and pricing) and the risk (from the failure modes in Section 6).

An LLM call is **cost-effective** when the task genuinely needs language understanding or generation, the volume is manageable, no cheaper deterministic method would do, and the output’s failure modes are tolerable or verifiable. It is **wasteful** when a rule, lookup, or smaller model would do the same job — or when the failure modes in Section 6 make the output untrustworthy for the use at hand, so that you cannot rely on it anyway.

**Worked example — Two decisions. Cost-effective:** summarising open-ended customer feedback into themes. There is no rule that does this; the input is genuinely natural language; an occasional imperfect summary is acceptable; and a human reads the result.

**Wasteful:** extracting the total from an invoice whose layout is fixed. A template or regular expression does it deterministically, for free, with no hallucination risk. An LLM here costs money to introduce an error the simpler method cannot make.

The decision procedure, then: estimate the cost (token count  $\times$  price, summed over the expected volume), weigh it against the value produced and the cost of an error, check the failure modes are acceptable, and choose the **cheapest model and simplest method** that meet the requirement. Often the right answer is a small model, or no model at all.